LESSON 26

门牌号 *p 那个盒子

指针入门

学完以后，能记下一个变量住在哪里，再顺着这个地址找到它、改它。

& 写在变量前面，又是什么意思？

where.cpp（只阅读）

```
5       int x = 5;
6       cout << x << endl;
7       cout << &x << endl;
```

屏幕上，比如

```
5
0x7ffd5e8c
```

每个变量在内存里都有一个住的地方，像门牌号，叫**地址**。&x 就是 x 的地址。

第 25 课的 & 写在类型后面（int &x），是引用；今天的 & 写在变量前面（&x），是取地址。

地址每台电脑、每次运行都可能不一样，不用记它是多少。

指针：一个专门存地址的变量

pointer.cpp

```cpp
    int x = 0;
    cin >> x;
    int *p = &x;
    cout << *p << endl;
    *p = *p + 1;
    cout << x << endl;
```

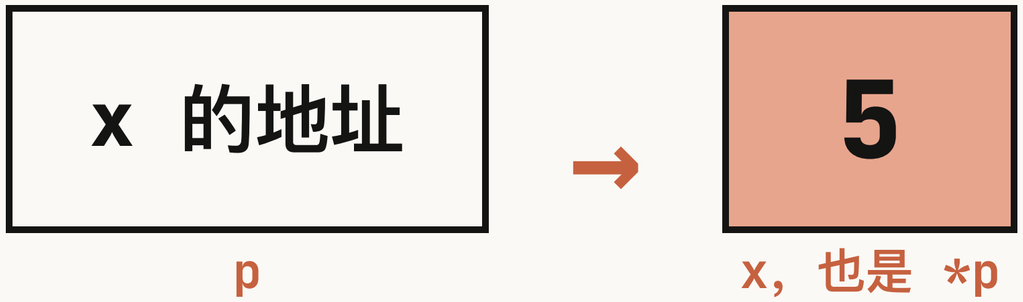
输入 5

```
5
6
```

*p 是顺着地址找到的那个盒子，也就是 x。改 *p 就是改 x。

int *p 准备一个存 int 地址的变量。p = &x，就说 p **指向** x。

p 是门牌号，*p 是那个盒子



`&x`	x 住在哪里，取地址
`int *p`	写在类型里：p 是存地址的
`*p`	写在 p 前面：顺着地址找到那个盒子

小测：输出几？

```
int x = 3; int *p = &x; *p = 7; cout << x;
```

1 3

2 7

3 x 的地址

4 报错

答对了！p 指向 x，*p = 7 把 x 改成了 7。

动手改一改

从 pointer 开始。全程不直接写 x =，只通过 p 改。

通过 p 变 3 倍

读入一个整数 x，让 p 指向 x，通过 p 把 x 变成原来的 3 倍，最后输出 x。

输入 5

```
15
```

输入 -4

```
-12
```

指针可以换着指：指向大的那个

pick.cpp

```cpp
    int a = 0, b = 0;
    cin >> a >> b;
    int *p = &a;
    if (b > a) {
        p = &b;
    }
    cout << *p << endl;
```

输入 3 8

```
8
```

先指向 a。b 更大，就把 p 改成 b 的地址。

p = &b 改的是 p 指向谁，a、b 里的数都没动。

猜一猜：a 和 b 各是几？

move.cpp（只阅读）

```
5       int a = 1, b = 2;
6       int *p = &a;
7       p = &b;
8       *p = 9;
9       cout << a << " " << b << endl;
```

屏幕上

```
1 9
```

p 先指向 a，又改成指向 b。*p = 9 的时候，p 指着的是 b。改到谁，要看那一刻 p 指着谁。

动手改一改

从 pick 开始。p 已经指向大的那个了。

把大的清零

读入两个整数，通过 p 把大的那个改成 0，再输出 a 和 b，中间空一格。一样大时改 a。

输入 3 8

```
3 0
```

输入 4 4

```
0 4
```

把地址交给函数，也能改外面

swap_ptr.cpp

```
4 void swapPtr(int *a, int *b) {
5     int temp = *a;
6     *a = *b;
7     *b = temp;
8 }
```

swap_ptr.cpp

```
13     swapPtr(&x, &y);
14     cout << x << " " << y << endl;
```

输入 5 3

```
3 5
```

调用时交的是地址 &x、&y。函数里用 *a、*b 找到外面那两个盒子。

第 25 课的引用和今天的指针，做的是同一件事

`int &a`	引用：函数里直接写 a
`swapTwo(x, y)`	引用：调用时照常写变量
`int *a`	指针：函数里要写 *a
`swapPtr(&x, &y)`	指针：调用时要交地址

只是想让函数改外面的变量，用引用更省事。指针能换着指，下一页就看它和数组的关系。

数组名当地址用，就是第一个盒子的地址

array.cpp（只阅读）

```
5       int a[3] = {4, 7, 2};
6       int *p = a;
7       cout << *p << endl;
8       cout << *(p + 1) << endl;
9       cout << *(a + 2) << endl;
```

屏幕上

```
4
7
2
```

数组名 a 当地址用，就是 a[0] 的地址，所以 int *p = a; 不用写 &。
p + 1 往后挪一个盒子，*(p + 1) 就是 a[1]。

第 22 课的 sort(a, a + n)：a 是第一个盒子，a + n 是最后一个后面那一格。
平时取数还是写 a[i]，更清楚。

读报错：p 要存地址，给了一个数

pointer.cpp（只阅读）

```
 7      int *p = x;
```

抓住两个词：'int *' 和 'int'，一个是地址，一个是数。

```
main.cpp:7:10: error: cannot initialize a variable of type 'int *'
with an lvalue of type 'int'
```

x 是一个数，p 只能存地址。要写 int *p = &x;。

找错：哪一行少了一个 *？

zero.cpp（只阅读）

```
1       int *p = &a;
2       if (b < a) {
3           p = &b;
4       }
5       p = 0;
6       cout << a << " " << b << endl;
```

这段想把小的那个改成 0。
输入 3 8，应该是 0 8，屏幕上却是 3 8。
点你认为错了的那一行。

点代码里的一行

自己挑战

最大的翻倍

读入三个整数 a b c（-1000 到 1000）。让一个指针走到最大的那个，通过指针把它变成 2 倍，再输出 a b c。一样大时改最前面那个。

输入 3 8 5，输出

3 16 5

输入 4 4 1，输出

8 4 1

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　先让 p 指向 a，就像第 10 课先把第一个当最大。

提示 2　b 比 *p 大，就让 p 指向 b；c 比 *p 大，就让 p 指向 c。

提示 3　最后 *p = *p * 2。用 4 4 1 试试，看改的是不是 a。

参考答案：最大的翻倍

double_max.cpp

```cpp
    int a = 0, b = 0, c = 0;
    cin >> a >> b >> c;
    int *p = &a;
    if (b > *p) {
        p = &b;
    }
    if (c > *p) {
        p = &c;
    }
    *p = *p * 2;
    cout << a << " " << b << " " << c << endl;
```

指针先指着 a，遇到更大的就改指它；最后 *p = *p * 2，改的是那个数本身。

指到谁，就改谁

用指针从几个变量里挑一个，再通过指针改它。主题你自己定。

先做到

按一个条件让指针指向其中一个，通过 `*p` 改它。

选你喜欢的

给少的那筐补水果、给落后的队加分、给最饿的宠物喂食，都可以。

还想更厉害

从三个里挑，或者挑完再挑一次。

没有标准答案。写之前说清想要的结果，再试试两个一样多。

和同桌换着玩

1

换着运行

运行同桌的程序，换几个数试试，再试试两个一样

2

说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

3

提一个想法

问问同桌：每一个 `*` 和 `&` 各是什么意思？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

说得清吗?

Q1　**`int &r = x` 和 `&x`，两个 `&` 一样吗?**

不一样：写在类型后面是引用，写在变量前面是取地址。

Q2　**`p` 和 `*p` 有什么不同?**

`p` 是地址；`*p` 是顺着地址找到的那个盒子。

Q3　**`p = &b` 和 `*p = 0`，各改了什么?**

前一个改 `p` 指向谁，后一个改 `p` 指着的那个数。

速查卡

`&x`	x 的地址
`int *p = &x;`	p 指向 x
`*p = *p + 1;`	通过 p 改 x
`p = &b;`	换成指向 b
`f(int *a)` `f(&x)`	把地址交给函数

这些后面再学

nullptr

什么都不指的指针，用之前要先检查

p++ 走数组

让指针一格一格往后走，把数组走一遍

new 和 delete

自己要盒子、还盒子，现在用 vector 就够了

今天学会指针：&x 是地址，p 存地址，*p 是那个盒子。

现在先记住：用 *p 之前，p 一定要指着一个真的盒子。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：枚举